ACADEMIC PROJECT · GENERATIVE AI

LoanMate AI

Complete Presentation & Viva Preparation Guide

Generative AI + Retrieval-Augmented Generation (RAG) Based Loan Assistant

Project Presentation Prep | Architecture | Technical Workflow | GenAI Concepts | RAG Implementation | Viva Questions | Demo Script | Future Improvements

PREPARED FOR VIVA & DEFENSE · VERIFIED AGAINST SOURCE CODE

Table of Contents

2 Explain LoanMate AI in 60 Seconds
3 Problem Statement
4 Project Objectives
5 What Exactly Did We Build?
6 Complete System Architecture
7 RAG — The Heart of LoanMate AI
8 RAG Pipeline — Step by Step
9 Embeddings Explained
10 Vector Store
11 Retriever
12 RAG Service (Orchestration)
13 File-by-File Code Architecture
14 End-to-End Query Flow
15 Where Is Generative AI Used?
16 GenAI Concepts Used in the Project
17 RAG vs Fine-tuning
18 RAG vs Normal Database
19 RAG vs Direct LLM
20 Hallucination
21 Prompting & Grounding
22 Data Flow Diagram
23 Technology Stack
24 Why the Project Is Modular
25 Security / Reliability Considerations
26 Limitations
27 Future Scope
28 Teacher Questions — Basic (25+)
29 Teacher Questions — Technical (30+)
30 Teacher Trap Questions
31 Demo Script
32 Live Architecture Explanation Script
33 Explain to a Non-Technical Teacher (Library Analogy)
34 Terminology Cheat Sheet
35 30-Second Answers
36 Final Presentation Story
37 Cheat Sheet Before Entering Class
38 Details I Must Verify From My Code

42 Mock Viva (20+ Exchanges)

43 If You Remember Only 10 Things

2. Explain LoanMate AI in 60 Seconds

🎤 **SAY THIS — Full 60-second version**

"LoanMate AI is a voice-based loan assistant I built for the BFSI — banking, financial services and insurance — domain. Instead of a customer logging into a banking app and hunting through menus to find their EMI or outstanding balance, they just have a live spoken conversation with LoanMate, in English, Hindi, or Hinglish.

The core of the system is a Generative AI model — Google's Gemini Live — which listens to the customer's speech directly and generates a natural spoken reply, in real time, over one continuous audio session. But I didn't want the model just guessing answers from its own training data, especially for a financial product. So the model is agentic: it doesn't answer numbers itself — it decides, per question, whether it needs to call a backend tool. For account-specific things like 'what's my EMI', it calls a deterministic Python function that reads the real customer record, so the number is never invented.

For general questions that aren't about one customer's account — 'what documents do I need for a personal loan', 'what's your prepayment policy' — there's no single customer record to look up. That's where I used Retrieval-Augmented Generation, RAG. I built a small knowledge base of 11 markdown documents — loan products, policies, FAQs, required documents — split them into 75 semantic chunks, converted each chunk into a numerical embedding using a Gemini embedding model, and stored those vectors in a local FAISS vector index. When the agent decides a question is a knowledge question, it calls a tool called `search_loan_knowledge`, which embeds the question, searches that FAISS index for the most similar chunks, and — only if they're relevant enough — hands that real text back to the model so it can answer using our actual documents instead of guessing.

So end to end: customer speaks → Gemini Live understands it → decides tool vs RAG vs direct answer → if RAG, the question is embedded, matched against our vector index, and the top matching chunks come back → Gemini uses that grounded text to generate a natural spoken answer → the customer hears it, and can interrupt at any point mid-sentence."

Ultra-short version — 20 seconds

🎤 **SAY THIS**

"LoanMate AI is a voice assistant for loan and EMI queries. It uses Gemini Live for realtime conversation and agentic tool-calling so numbers always come from real data, not the model's guess. For general policy and product questions, it uses a RAG pipeline — our documents are embedded and stored in a FAISS vector index, and the relevant chunks are retrieved and given to the model before it answers, so those answers are grounded in our own knowledge base too."

If the teacher says: "Explain your project in simple words"

🎤 **SAY THIS**

"It's like a smart phone-call bot for a bank. You call it, ask about your loan in your own words, and it replies by voice like a real person. For your personal details it looks up your real account. For general questions about loan policies, it first searches our own written documents to find the correct answer, and only then replies — so it's not making things up, it's reading from real material first."

3. Problem Statement

Loan-related information is **conceptually simple but operationally tedious to access**. A customer wanting to know their EMI, due date, or eligibility for prepayment has to log in, navigate a dashboard, and interpret banking terminology — for a question whose answer is one sentence.

Real-world problems this addresses

- Loan/EMI information is scattered across dashboard widgets, PDFs, and policy pages.
- Customers often don't know banking terminology (foreclosure, tenure, FOIR) well enough to find the right screen.
- Traditional apps give **static**, pre-built screens — they can't answer a free-form natural-language question.
- Many users are more comfortable **speaking** than typing, especially in Hindi/English mixed speech (Hinglish).
- If you simply point a generic LLM at this problem with no grounding, it may answer fluently but **incorrectly** — inventing a policy, a number, or a rule that doesn't exist. This is unacceptable for a financial product.

Problem → Traditional approach → Our approach → Benefit

Problem	Traditional approach	LoanMate AI approach	Benefit
Finding your EMI/balance	Login → dashboard → locate widget	Ask by voice; a deterministic tool (`get_emi` , `get_outstanding_balance`) reads the real record	One spoken question instead of multi-step navigation
Understanding loan policies (prepayment, foreclosure, documents)	Search a PDF/FAQ page manually	RAG pipeline retrieves the relevant document chunk and the model explains it in natural language	Answers grounded in our real documents, explained conversationally
Language barrier (Hindi/English mix)	UI is usually English-only	Gemini Live natively understands/generates English, Hindi, and Hinglish in one session	No separate translation layer needed
Risk of an AI "just making things up"	N/A (traditional apps don't generate text)	Deterministic tools for numbers + RAG grounding for knowledge questions, both explicit, both traceable	Lower hallucination risk than a bare LLM (not eliminated — see §20)

> ⚠️ **IMPORTANT — say this precisely**
>
> Don't claim RAG "solves hallucination completely." The honest, correct claim is: RAG **reduces** the chance of unsupported answers on knowledge questions by giving the model real text to work from, and an explicit relevance threshold (0.65) prevents the model from getting an answer at all when nothing in the knowledge base is actually relevant.

4. Project Objectives

1. Build an intelligent, conversational loan-assistance system — not a static dashboard.
2. Provide realtime, natural spoken interaction (not a turn-based chatbot).
3. Use Generative AI (Gemini Live) as the conversational core, so replies are dynamically generated, not template-selected.
4. Ground customer-specific answers in real data via deterministic tool calls — never let the model invent a number.
5. Ground general knowledge/policy answers using Retrieval-Augmented Generation over a real local knowledge base.
6. Reduce (not claim to eliminate) hallucination risk compared to a bare, ungrounded LLM.
7. Keep the RAG pipeline modular — loading, chunking, embedding, storing, retrieving, and orchestration are separate files with single responsibilities.
8. Make the knowledge base easy to update — plain Markdown files, re-indexed with one offline command (`python -m backend.rag.ingest`).
9. Support English, Hindi, and Hinglish in the same conversation, without a separate translation layer.
10. Support both inbound ("Call LoanMate") and simulated outbound ("Bank is Calling You") conversations.

5. What Exactly Did We Build?

Frontend

A voice-first Next.js/React web app (`frontend/web/`) — deliberately not a banking dashboard. It shows a voice orb, call controls (mute/end), a language picker, and call state (connecting/listening/speaking). It connects to LiveKit over WebRTC using `@livekit/components-react`. A legacy Streamlit app (`frontend/app.py`) also exists for a text/voice chat demo and calculator widgets, kept as a fallback reference.

Backend

A FastAPI application (`backend/main.py`) that issues LiveKit access tokens, validates the requested customer against mock data, and exposes deterministic REST endpoints for loan/EMI calculations and customer lookup. For the primary voice path, FastAPI itself never touches audio — it only hands out a token.

AI / LLM layer

Google's **Gemini Live** (`gemini-3.8-live`) is the realtime conversational model — a single audio-in/audio-out session that does understanding, reasoning about which tool to call, and speech generation together. A separate, smaller **Gemini embedding model** (`gemini-embedding-2`) is used purely for RAG — it never talks to the user, it only turns text into vectors.

Knowledge base

11 Markdown files under `backend/knowledge/`, organized into `loan_products/`, `policies/`, `faq/`, and `documents/`. They are explicitly written as fictional demo content for a fictional "LoanMate Bank," not a real institution's actual policy text.

RAG pipeline

Six Python modules under `backend/rag/` (`document_loader.py`, `chunker.py`, `embeddings.py`, `vector_store.py`, `retriever.py`, `rag_service.py`) plus an offline indexing script (`ingest.py`) and an evaluation script (`eval_set.py`). Together they load the Markdown files, split them into semantic chunks, embed each chunk, store the vectors in a local FAISS index, and search that index at query time.

Component table

Component	Purpose	Why it exists
Next.js frontend	Voice-first UI; renders call state and connects to LiveKit	User needs a real interface to start/control a call
FastAPI backend	Issues LiveKit tokens; validates customer; exposes REST loan endpoints	A stateless, simple API layer decoupled from the always-on voice worker
LiveKit Agents worker (realtime_agent.py)	Holds the actual Gemini Live connection; defines all 10 tools	LiveKit's architecture requires a persistent, separately-registered worker process, not something a request/response API can host
Gemini Live (gemini-3.8-live)	Realtime conversational + voice generation model	Enables natural, interruptible, audio-native conversation instead of a record→transcribe→reply cascade

9 deterministic tools	Read real customer/loan data or compute a result	Guarantees every financial figure is real, never model-invented
RAG pipeline (backend/rag/)	Embeds and searches the knowledge base; grounds general policy/product answers	Gives the model a way to answer questions no single customer record can answer
Knowledge base (backend/knowledge/*.md)	Source-of-truth text for products, policies, FAQs, documents	Something concrete and inspectable to retrieve from, instead of the model's own unverifiable knowledge

6. Complete System Architecture

There are genuinely **two separate flows** in this project — do not blur them together when explaining.

Online / Question-Answering Flow (what happens during a live call)

```
User (speaks)
  ↓
Next.js Frontend  (voice orb, LiveKit React SDK)
  ↓  WebRTC audio
LiveKit Cloud  (media transport + agent auto-dispatch)
  ↓
LoanMate Realtime Agent worker  (realtime_agent.py, LiveKit Agents)
  ↓  streams audio into
Gemini Live (gemini-3.8-live)  — understands speech, decides: tool? RAG? direct?
  ↓ (if a knowledge/policy question)
search_loan_knowledge tool  →  RAGService.search()
  ↓
Retriever.retrieve()  →  embed_query()  (gemini-embedding-2, 768-dim)
  ↓
FAISS vector index (IndexFlatIP)  — cosine-similarity search, top_k = 3
  ↓
Chunks scoring ≥ 0.65 returned, with [source: file.md — Section] tags
  ↓
Relevant knowledge chunks handed back into the live Gemini session (as the tool's result)
  ↓
Gemini Live composes a grounded, natural spoken answer
  ↓
LiveKit Cloud → Frontend → User hears the answer
```

Offline / Knowledge-Indexing Flow (run once, before the app is used — `python -m backend.rag.ingest`)

```
backend/knowledge/*.md  (11 files)
  ↓
document_loader.py  → load_documents()  → RawDocument objects
  ↓
chunker.py  → chunk_documents()  → 75 Chunk objects (split on "## " section headings)
  ↓
embeddings.py  → embed_document()  (gemini-embedding-2, task_type=RETRIEVAL_DOCUMENT)
  ↓
vector_store.py  → VectorStore.build()  → FAISS IndexFlatIP, 75 × 768
  ↓
VectorStore.save()  → backend/rag/vector_store/loanmate.index + metadata.json
```

> ⚠️ **IMPORTANT**
>
> The running application (FastAPI + the realtime worker) **never rebuilds the index at startup**. It only *loads* the two saved files (`loanmate.index` , `metadata.json`). This means a normal run of LoanMate AI makes **zero** embedding API calls unless the agent actually calls `search_loan_knowledge` during a conversation (to embed the query). Re-indexing only happens when you deliberately run `ingest.py` after editing the knowledge base.

Why RAG is a *tool*, not a step that runs on every message

A common misconception in generic RAG explanations is that retrieval happens on every single user turn. That is **not** how LoanMate AI works. Gemini Live decides, per question, whether it's (a) a customer-specific fact → call one of the 6 data-lookup tools, (b) a calculation → call one of the 3 calculator tools, (c) a general knowledge/policy question → call `search_loan_knowledge` (RAG), or (d) something answerable directly from context already in the prompt (like the customer's own name). RAG only fires for case (c). This is **agentic RAG** — retrieval is one capability among several that the model chooses to invoke, not an unconditional pipeline stage.

7. Retrieval-Augmented Generation (RAG) — The Heart of LoanMate AI

> ⚠️ **ONE-SENTENCE DEFINITION**
> RAG allows the LLM to retrieve relevant information from an external knowledge source **before** generating an answer, instead of answering purely from what it already learned during pretraining.

Without RAG

```
User question → LLM → Answer
```

The problem: the LLM only relies on whatever it learned during training (plus whatever is in the immediate conversation). It has never seen LoanMate's specific 11 documents. If asked "what is LoanMate's prepayment charge," a bare LLM has no way to know the real answer — it can only guess something that *sounds* plausible for a generic bank. That guess might be wrong. This is hallucination.

With RAG

```
User question
  → Convert question into an embedding (gemini-embedding-2, task_type=RETRIEVAL_QUERY)
  → Search the FAISS vector store for similar chunk embeddings
  → Keep only chunks with cosine similarity ≥ 0.65
  → Hand the matched chunk text (with source/section tags) back to the LLM
  → LLM generates an answer grounded in that real text
```

Why this matters specifically for LoanMate AI

- The 9 financial tools only know **one customer's own account**. They cannot answer "what documents do I need for a personal loan" — that's not looked up from a customer record, it's a general policy fact.
- RAG closes exactly that gap: it gives the model a way to search real reference text (our own written policy/product/FAQ documents) instead of guessing about "LoanMate Bank"'s made-up policies.
- Because the knowledge base is small (11 files) and explicit, every answer produced this way can be traced back to a specific file and section — `[source: prepayment_policy.md — Charges]`, for example.

8. RAG Pipeline — Detailed Step-by-Step

Step 1 — Knowledge Documents

The knowledge base is 11 Markdown files under `backend/knowledge/`, split across four category folders:

Folder	Files	Topic area
`loan_products/`	personal_loan.md, home_loan.md, education_loan.md	What each loan product is, rates, eligibility
`policies/`	prepayment_policy.md, foreclosure_policy.md, missed_emi_policy.md, eligibility_policy.md	Rules that apply across products
`faq/`	loan_faq.md, emi_faq.md, general_banking_faq.md	Common customer questions
`documents/`	required_documents.md	What paperwork is needed per loan type

Why Markdown? Each file already has meaningful `##` section headings (Overview, Eligibility Criteria, Charges, Frequently Asked Questions, etc.). That structure is not decorative — the chunker in Step 3 uses those exact headings to split the file into topically coherent pieces and to label each chunk with a real section name. Plain `.txt` would not give the chunker that structure "for free."

> 🔍 **VERIFY FROM CODE**
> The exact wording inside each of the 11 files (e.g. specific interest-rate numbers, specific charge percentages beyond the "2–5% of outstanding principal" figure already confirmed in a verified live test transcript) — re-read the actual .md files before quoting exact figures live in the viva.

Step 2 — Document Loading (`document_loader.py`)

What it does: recursively finds every `.md` file under `backend/knowledge/`, reads its raw text, and builds one `RawDocument` object per file with:

- `source_file` — the filename, e.g. `personal_loan.md`
- `document_type` — taken from the parent folder name, e.g. `loan_products`
- `topic` — the filename without extension, e.g. `personal_loan`
- `title` — the first `#` heading found in the file (falls back to the topic if none)
- `text` — the full raw Markdown text

Why metadata matters: this is plain filesystem convention — folder name and filename — not a classifier model. But it's exactly what later lets a retrieved chunk say "this came from `policies/prepayment_policy.md`," which is what makes an answer traceable rather than a black box.

Step 3 — Chunking (`chunker.py`)

Why split documents at all? Sending an entire document to an LLM (or embedding an entire document as one vector) is inefficient and imprecise: a whole file mixes multiple topics into one embedding, which makes semantic search blurry — a query about "charges" would also match irrelevant parts of the same file about "eligibility." Smaller, focused chunks let the retriever match a query to the *specific* paragraph that actually answers it.

How LoanMate AI chunks — this is a semantic split, not a blind fixed-character split: the chunker splits each document on its own `##` section headings (Overview, Eligibility Criteria, Charges, Frequently Asked Questions, etc.), so each chunk is one coherent section, not an arbitrary character-count slice that could cut a sentence in half. A

defensive fallback (`_MAX_CHUNK_CHARS = 1200`) further splits any single section that is unusually long — on paragraph boundaries — but this does not currently trigger on any of the 11 knowledge files, since every section is a few short paragraphs.

```
Example (illustrative — actual wording is in the real files, not reproduced verbatim here):

  "# Personal Loan
   ## Overview
   ...intro paragraph...
   ## Eligibility Criteria
   ...eligibility rules...
   ## Interest Rate & Charges
   ...rate/charges info..."

        ↓ chunked into

  Chunk 1  [section: Overview]
  Chunk 2  [section: Eligibility Criteria]
  Chunk 3  [section: Interest Rate & Charges]
  ...
```

Across all 11 documents this produces **75 chunks**. Each chunk carries full metadata (`chunk_id` , `source_file` , `document_type` , `topic` , `section`) and its text is prefixed with `"{title} — {section name}"` so the embedding also "sees" what document/section it belongs to.

> ⚠️ **IMPORTANT**
> There is no separate "chunk overlap" parameter in this project (a common RAG technique where consecutive chunks share a few overlapping sentences). LoanMate AI's chunks are non-overlapping section splits — overlap wasn't needed because each section is already a coherent, self-contained unit.

Why smaller chunks improve retrieval

A focused chunk produces a focused embedding. When a user asks "what happens if I miss an EMI," the query embedding needs to be close, in vector space, to a chunk that is *specifically about* missed EMIs — not diluted by unrelated eligibility or documentation text sitting in the same file.

9. Embeddings — Explained Like a Teacher May Ask

What is an embedding?

An embedding converts a piece of text into a fixed-length list of numbers (a **vector**) that represents the text's *meaning*, not its exact spelling. Two pieces of text with similar meaning end up as vectors that are close together in that numerical space, even if they don't share the same words.

```
Text  →  Embedding Model  →  Vector  (a list of numbers, e.g. [0.021, -0.114, 0.502, ... ]  — 768 numbers long in this project)
```

Example — semantic similarity, not keyword matching

"personal loan eligibility" and "who can apply for a personal loan?" share almost no exact words in common, but they *mean* almost the same thing. A good embedding model places their vectors close together, because it was trained to represent meaning — not to do string matching.

> 🎤 **SAY THIS**
> "An embedding turns text into a vector of numbers that captures meaning. Two sentences that mean similar things end up with similar vectors, even if the actual words are completely different. That's what lets our system match 'what happens if I miss an EMI' to a document section titled 'Missed EMI Consequences' — even though the wording doesn't match exactly."

What LoanMate AI actually uses

Detail	Value (verified from `backend/rag/embeddings.py`)
Embedding model	`gemini-embedding-2`, via the current `google-genai` SDK (not the deprecated `google-generativeai` SDK the legacy text path uses)
Output dimensionality	Truncated to **768 dimensions** (a supported Matryoshka-style truncation of the model's native 3072-dim output — chosen to keep the local FAISS index small)
Task types	Document chunks are embedded with `task_type="RETRIEVAL_DOCUMENT"`; the user's query is embedded with `task_type="RETRIEVAL_QUERY"` — Gemini's documented mechanism for asymmetric retrieval embeddings, so a short question and a longer passage compare well even though they're different shapes of text
Batching	Chunks are embedded **one at a time**, not in a batch — batch embedding was tested and found to silently return fewer vectors than inputs, which would have misaligned chunk text with metadata

Why semantic similarity beats exact keyword matching here

A customer might ask "Prepayment karne pe koi charges hain kya?" (Hinglish for "are there charges for prepaying"). No exact keyword match exists against an English document titled "Prepayment Policy — Charges." But because Gemini's embedding model captures meaning across languages/phrasing, the query's embedding still lands close to the right chunk's embedding. This was verified live — that exact Hinglish question correctly retrieved `prepayment_policy.md`.

Vector, dimensions, cosine similarity — plain definitions

- **Vector** — an ordered list of numbers; here, 768 numbers per piece of text.
- **Dimensions** — how many numbers are in that list (768 in this project).

- **Cosine similarity** — a measure of how "close in direction" two vectors are, from -1 (opposite meaning) to 1 (identical meaning). LoanMate AI computes this via a mathematically equivalent trick: vectors are normalized to unit length, and then inner product = cosine similarity (see §10).

> 🔍 **VERIFY FROM CODE**
> If asked "why 768 and not the full 3072," the honest answer already confirmed in project documentation is: it keeps the local FAISS index smaller/faster while still being a supported truncation of the model's native output — this was confirmed working via a real API call, not merely assumed.

10. Vector Store

What is a vector store?

A vector store is a database (here, a local, file-based one) whose job is to store many vectors and, given a new "query vector," quickly find the stored vectors most similar to it. It is optimized for *similarity search*, not exact-match lookups.

```
Document chunk → embedding (768-dim vector) → stored in the vector store, alongside its text + metadata

Later, at query time:
User question → embedding (768-dim vector) → similarity search against every stored vector → most relevant chunks returned
```

What LoanMate AI actually uses

Detail	Value (verified from `backend/rag/vector_store.py`)
Library	FAISS (`faiss-cpu`) — a real, open-source vector search library, not a hand-rolled substitute
Index type	`IndexFlatIP` — a **flat, exact** (brute-force) index, over L2-normalized vectors, so inner product = cosine similarity. This is **not** an approximate/ANN index like HNSW or IVF.
Why exact, not approximate	At only 75 chunks, an exact search is already sub-millisecond. An ANN index would trade a small amount of accuracy for no real speed benefit at this tiny scale — the honest, correct choice at this size is exact search.
Storage	Two local files: `backend/rag/vector_store/loanmate.index` (the FAISS index) and `metadata.json` (chunk text + metadata, row-aligned with the index). No external/hosted vector database, no paid service.
Size	75 vectors × 768 dimensions

Traditional database search vs. vector similarity search

Traditional (SQL-style) search	Vector similarity search
Exact/structured match: `WHERE category = 'personal_loan'`	Semantic match: "find text whose *meaning* is close to this question"
Requires the query to match a known field/value exactly (or via a fixed pattern like LIKE '%word%')	Works even when the user's wording is completely different from the document's wording
Good for structured records (customer_id, loan_id, amounts)	Good for free-form natural-language questions
Used in LoanMate AI for: customer/loan lookups (`mock_data.json`)	Used in LoanMate AI for: the knowledge-base RAG tool only

> 🤔 **IF TEACHER ASKS WHY — Why a vector store here, not just the existing JSON files?**
>
> The 11 knowledge documents are free-form paragraphs, not structured rows. There is no fixed "field" to query like `WHERE topic = X` that would reliably capture "what happens if I miss an EMI" in every possible phrasing. A vector store lets the system match on *meaning*, which is what a natural-language question needs.

11. Retriever

What does the retriever do?

`backend/rag/retriever.py` is the layer that turns "a user's question" into "a short list of relevant, trustworthy chunks." Concretely, `Retriever.retrieve(query, top_k=3)`:

1. Embeds the query using `embed_query()` (task_type= `RETRIEVAL_QUERY`).
2. Searches the FAISS index for the `top_k` most similar chunk vectors (**default** `top_k = 3`).
3. Filters out any result scoring below `MIN_RELEVANCE_SCORE = 0.65`.
4. Returns the surviving chunks as `RetrievedChunk` objects (text, score, source_file, document_type, topic, section).

Key concepts

- **Top-k retrieval** — instead of returning every chunk in the whole knowledge base, only the `k` highest-scoring ones are returned (here, up to 3). This keeps the context handed to the LLM small and focused.
- **Similarity score** — the cosine similarity (0 to 1 in practice here) between the query vector and a chunk vector. Higher = more semantically related.
- **Relevance threshold** — a hard cutoff below which a result is discarded entirely, even if it's the "best available" match.

Where 0.65 came from — this is not a guessed number

The threshold was derived empirically by running `backend/rag/eval_set.py`'s on-topic test questions and a set of clearly off-topic questions against the real, built FAISS index and inspecting the actual score distribution:

Query type	Best-match score range (measured)
On-topic (9 test queries, English/Hindi/Hinglish)	0.713 – 0.860
Off-topic (4 clearly unrelated queries, e.g. "tell me a joke")	0.525 – 0.591

`0.65` sits in the empirical gap between these two measured clusters. Off-topic queries still get *some* positive similarity score (never near-zero) because every chunk shares generic banking vocabulary — which is exactly why a threshold, not just "return the best match no matter what," is necessary.

> ⚠️ **IMPORTANT**
> This threshold is what stops the agent from receiving (and therefore narrating) a retrieval result for a genuinely off-topic question like "what's the weather today." In a verified live test, the agent honestly declined that question instead of inventing an answer.

12. RAG Service (Orchestration Layer)

`backend/rag/rag_service.py` is the layer **the realtime agent actually calls** — it is what backs the `search_loan_knowledge` tool.

```
User Query
  ↓
Retriever.retrieve(query, top_k=3)
  ↓
Relevant Context  (chunks scoring ≥ 0.65, or none)
  ↓
Prompt / tool-result Construction  (RAGService formats a compact, source-attributed text block)
  ↓
Handed back into the live Gemini session as the search_loan_knowledge tool's return value
  ↓
Gemini Live generates a grounded spoken response
```

What it does, precisely

- Lazily loads the `Retriever` (and therefore the FAISS index) once — not on every request, and not on every FastAPI/worker startup path that doesn't need it.
- Calls `retriever.retrieve(query, top_k)`.
- If nothing survives the relevance threshold, returns an explicit message: "*No sufficiently relevant information was found in the LoanMate knowledge base.*" — the model is never left to fill that gap with an invented answer.
- Otherwise, formats each surviving chunk as `"[source: {file} — {section}]\n{chunk text}"` and joins them — never raw embedding vectors, never entire documents.

Why this is a separate service (not folded into the retriever or the agent)

Separating "search and score chunks" (retriever.py) from "format the agent-facing answer, including the explicit no-match message" (rag_service.py) means the retriever stays a general-purpose, reusable semantic-search component, while the agent-facing formatting/wording — which is specific to how the LiveKit tool should present its result — lives in exactly one place. If the "no match" wording needs to change, or the response format needs to change, only `rag_service.py` needs editing.

13. File-by-File Code Architecture

File	Responsibility	Input	Output	Why it matters
`document_loader.py`	Load every .md file from the knowledge base	Folder path (`backend/knowledge/`)	List of `RawDocument` (file text + metadata)	Single source of raw text; everything downstream depends on consistent metadata here
`chunker.py`	Split each document into semantic (section-based) chunks	List of `RawDocument`	List of `Chunk` (75 total)	Chunk granularity directly controls retrieval precision
`embeddings.py`	Convert text (chunk or query) into a 768-dim vector via Gemini	A string (chunk text or user query)	List of 768 floats	The actual "meaning → numbers" step that makes semantic search possible
`vector_store.py`	Build, save, load, and search a local FAISS index	Vectors + chunk metadata (build); a query vector (search)	A saved index (build); ranked (score, chunk) pairs (search)	Makes similarity search fast and persistent across runs
`retriever.py`	High-level semantic search: embed query → search → filter by relevance	A user's question (string)	List of `RetrievedChunk` , above threshold, ranked	The single place that decides "is this actually relevant?"
`rag_service.py`	Agent-facing entry point; formats results or returns an explicit "no match" message	A user's question (string)	A compact, source-attributed text block (string)	What the `search_loan_knowledge` tool actually calls
`ingest.py`	Offline pipeline runner: load → chunk → embed → build → save	Nothing (reads the knowledge folder directly)	Two files: `loanmate.index` , `metadata.json`	The only place embedding API calls happen for indexing; run manually after editing knowledge files
`backend/knowledge/`	Not code — the actual source content (11 .md files, 4 folders)	—	—	What gets retrieved; editing these + re-running `ingest.py` is the entire "update the knowledge base" workflow

For every file — simple, technical, how it connects, what to say

document_loader.py

Simple: reads all the knowledge files off disk and figures out which folder/file each piece of text came from.
Technical: recursively globs `*.md` under `backend/knowledge/`, extracts the first `#` heading as the title, and returns frozen `RawDocument` dataclass instances.
Connects to: feeds `chunker.py`.
Say this: "It's the first stage — it just loads the raw text and tags each file with its folder (document_type) and filename (topic), so later stages can attribute an answer to the right file."

chunker.py

Simple: cuts each document into smaller, topic-focused pieces instead of one giant blob.
Technical: splits on `##` markdown headings (semantic split), with a 1200-character paragraph-boundary fallback for unusually long sections; produces 75 `Chunk` objects with full metadata.
Connects to: feeds `embeddings.py` (via `ingest.py`) and carries metadata through to `rag_service.py`'s source citations.
Say this: "It splits on the document's own section headings, not a blind character count, so each chunk stays topically coherent and keeps a real section name attached."

embeddings.py

Simple: turns text into a list of numbers that represents its meaning.
Technical: wraps Google's `google-genai` SDK, model `gemini-embedding-2`, output truncated to 768 dimensions, using asymmetric `RETRIEVAL_DOCUMENT` / `RETRIEVAL_QUERY` task types.
Connects to: used by `ingest.py` (indexing) and `retriever.py` (query time).
Say this: "This is the only place text actually becomes a vector — everything before it is just text, everything after it is just numbers and search."

vector_store.py

Simple: a searchable filing system for all those number-vectors.
Technical: a local FAISS `IndexFlatIP` (exact, cosine-equivalent) over L2-normalized vectors, saved as two flat files.
Connects to: built once by `ingest.py`; loaded and searched by `retriever.py`.
Say this: "It's a real FAISS vector index, running locally — no external database, no paid service — chosen as an exact index because our knowledge base is small enough that exact search is already sub-millisecond."

retriever.py

Simple: takes a question, finds the best-matching pieces of knowledge, and throws away anything not actually relevant.
Technical: embeds the query, searches the index for top_k=3, filters by `MIN_RELEVANCE_SCORE=0.65` (empirically derived).
Connects to: called by `rag_service.py`.
Say this: "This is the component that decides what counts as 'relevant enough' — anything below a 0.65 cosine-similarity score gets discarded, which is what stops the system from confidently answering an off-topic question."

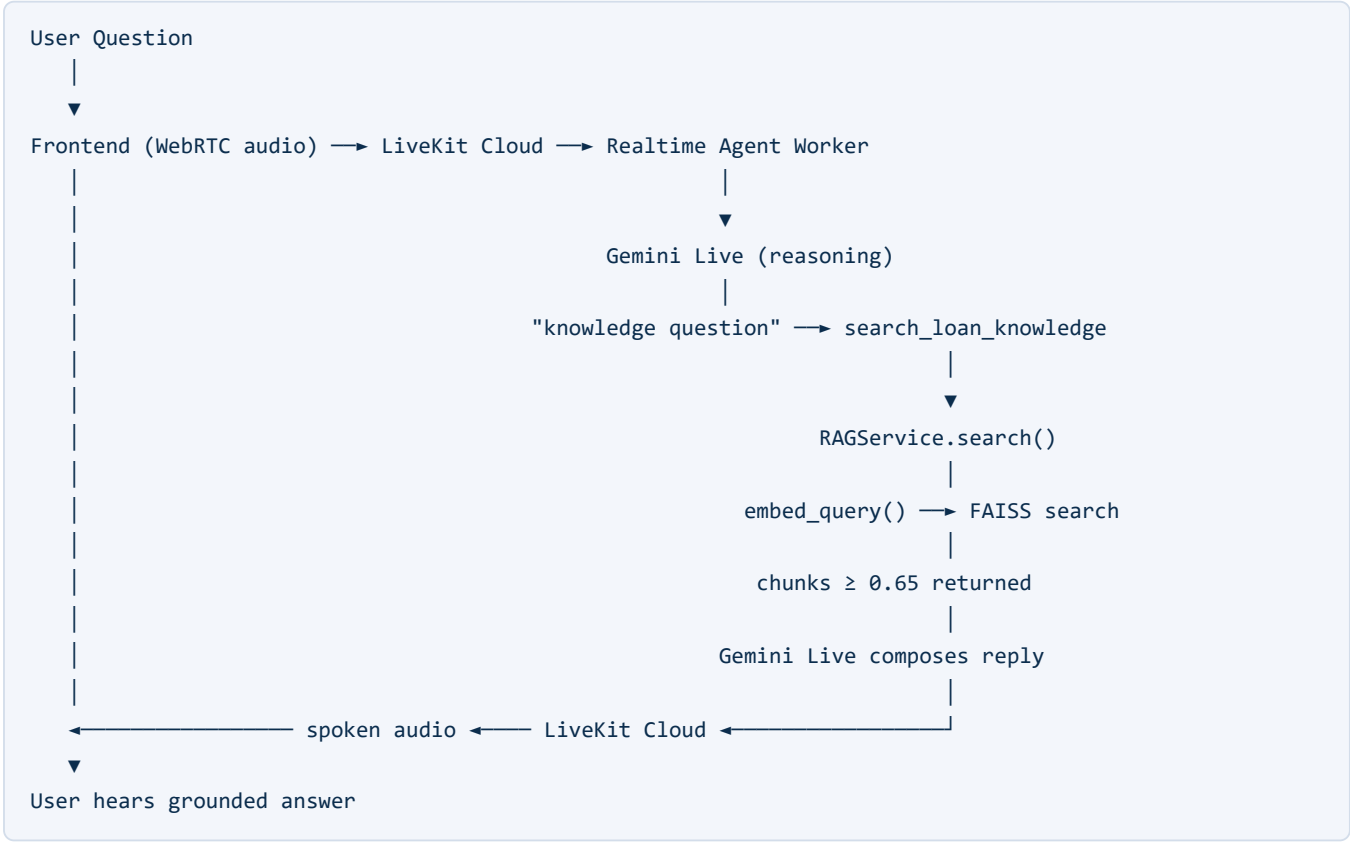
rag_service.py

Simple: packages the retrieved knowledge into something the AI agent can actually use and speak from.
Technical: lazily loads a singleton `Retriever`; formats results as source-attributed text blocks, or returns an explicit "not found" message.
Connects to: called directly by the `search_loan_knowledge` tool in `realtime_agent.py`.
Say this: "This is the orchestration layer — it's literally the function the voice agent calls when it decides a question needs the knowledge base."

14. End-to-End Query Flow

Example query: **"What are the eligibility requirements for a personal loan?"**

1. Customer speaks the question into the browser microphone during an active LoanMate call.
2. The frontend (Next.js + LiveKit React SDK) streams the live audio over WebRTC — this is not an HTTP request per question, it's a continuous media stream.
3. LiveKit Cloud relays the audio to the already-connected LoanMate Realtime Agent worker process.
4. The agent streams the audio directly into the open Gemini Live WebSocket session.
5. Gemini Live transcribes/understands the speech internally and reasons: "this is a general eligibility question, not this customer's own account data" → decides to call the `search_loan_knowledge` tool.
6. `search_loan_knowledge(query="eligibility requirements for a personal loan")` executes, calling `RAGService.search()`.
7. `Retriever.retrieve()` embeds the query via `embed_query()` (gemini-embedding-2, RETRIEVAL_QUERY task type) → a 768-dim vector.
8. The FAISS index is searched for the top 3 most similar chunk vectors (cosine similarity via normalized inner product).
9. Chunks scoring ≥ 0.65 survive — in practice for an eligibility-style question, this would likely retrieve from `eligibility_policy.md` and/or `personal_loan.md`.
10. `rag_service.py` formats the surviving chunks into a compact, source-attributed text block.
11. That text is returned as the tool's result, back into the live Gemini Live session.
12. Gemini Live generates a natural spoken answer using that retrieved text as grounding — it does not just repeat the raw chunk, it explains it conversationally.
13. The generated audio streams back through LiveKit Cloud to the browser.
14. The customer hears the answer, spoken naturally, grounded in the real `eligibility_policy.md` / `personal_loan.md` content.



15. Where Is Generative AI Used?

> ⚠️ **MOST IMPORTANT SECTION — say this well**
>
> The system is never doing `if question == X: answer Y`. There is no fixed lookup table of question→answer pairs for open-ended conversation. Every spoken reply the customer hears is **dynamically generated, natural language**, produced token-by-token by Gemini Live at the moment of the conversation — not selected from a pre-written script.

Input → Retrieval → Generation

Stage	What happens	Is this "AI generating text"?
Input	Customer's natural-language spoken question	No — this is just the user's speech, captured as audio
Retrieval	Relevant project knowledge is found via embedding similarity search (FAISS)	**No — this is search, not generation.** It finds existing text; it does not write anything new.
Generation	Gemini Live composes a new, natural-language sentence using the retrieved text (and/or a tool's returned number) as grounding	**Yes — this is where Generative AI happens.** The exact wording of the answer is generated fresh, every time.

> 🎤 **SAY THIS — if asked "where exactly have you used Generative AI?"**
>
> "Generative AI is Gemini Live itself — the model that listens to the customer's speech and produces a brand-new sentence of spoken language every single turn. It's used in two ways: first, for the actual conversational understanding and reply generation across the whole call — deciding what to say and how to say it. Second, specifically for RAG-grounded answers, the generation step is what turns three retrieved knowledge-base chunks into one natural, conversational explanation — the retrieval step only found the raw material; Gemini Live is what wrote the actual sentence the customer hears."

> 🚨 **TEACHER TRAP**
>
> "But isn't retrieval just search?" — Yes, correctly. Retrieval (FAISS + embeddings) is *not* Generative AI on its own; it's semantic search. It becomes part of a *Generative AI system* only because its output feeds a generative model that then writes the final answer. If you stopped after retrieval, you'd just have three chunks of raw markdown text — not an answer.

16. Generative AI Concepts Used in the Project

Large Language Models (LLMs)

Definition: a large neural network trained on huge amounts of text, able to understand and generate human language.
How it works generally: predicts the next most plausible piece of text given everything so far, at massive scale.
How LoanMate AI uses it: Gemini Live (`gemini-3.8-live`) is the LLM powering the entire conversation, including tool-calling decisions.
Say in viva: "Gemini Live is a large, pretrained language model that understands and generates natural spoken language in real time."

Prompting

Definition: the instructions and context given to an LLM to shape its behavior.
How it works generally: a system prompt sets persona/rules; user input and any retrieved context are added as the model's input.
How LoanMate AI uses it: `backend/voice_agent/prompts.py` builds the system instructions (persona, customer-context block, tool-usage rules, language rules) given to Gemini Live once per call.
Say in viva: "Prompting is how we tell the model who it is, what data it has, and what rules to follow — without changing the model itself."

Context Injection

Definition: putting relevant information directly into the model's input so it doesn't have to guess.
How it works generally: structured facts (like a customer's name) are inserted into the prompt as plain text.
How LoanMate AI uses it: the customer's name/ID/loan ID is injected once per call as a `CUSTOMER CONTEXT` block — this is context injection, distinct from retrieval, because it's a fixed lookup for one known customer, not a similarity search.
Say in viva: "Context injection gives the model known facts directly; retrieval searches for facts it doesn't already have."

Retrieval-Augmented Generation (RAG)

Definition: retrieving relevant external text before generation, so the answer is grounded in real material.
How it works generally: embed the query → search a vector store → hand matched text to the LLM.
How LoanMate AI uses it: the entire `backend/rag/` pipeline, exposed as the `search_loan_knowledge` tool.
Say in viva: "RAG is how we ground general knowledge/policy answers in our own 11 documents instead of the model's guesses."

Embeddings

Definition: numeric vector representations of text meaning.
How it works generally: an embedding model maps text to a fixed-length vector such that similar meanings → nearby vectors.
How LoanMate AI uses it: `gemini-embedding-2` , 768 dimensions, used for both knowledge chunks and live queries.
Say in viva: "Embeddings are what let us compare a question and a document section by meaning, not by exact words."

Semantic Search

Definition: finding text by meaning rather than exact keyword match.
How it works generally: compare the query's embedding against stored embeddings; return the closest ones.
How LoanMate AI uses it: the FAISS `IndexFlatIP` similarity search inside `vector_store.py`.
Say in viva: "Semantic search is what makes 'agar main do lakh prepay karun' correctly match our English-language prepayment policy document."

Vector Similarity

Definition: a numeric measure of how close two vectors are in meaning-space.
How it works generally: cosine similarity (or an equivalent normalized inner product) scores 0–1 (in this project's normalized setup).
How LoanMate AI uses it: normalized inner product in FAISS, filtered at `MIN_RELEVANCE_SCORE = 0.65`.
Say in viva: "Vector similarity is the actual number we threshold on to decide if a retrieved chunk is trustworthy enough to use."

Knowledge Grounding

Definition: tying a generated answer to a specific, real, checkable source.
How it works generally: pass retrieved source text into generation, optionally attach citations.
How LoanMate AI uses it: every RAG answer's underlying chunk carries a `[source: file.md — Section]` tag through the pipeline (kept in logs/transcripts for debugging, even if not always spoken aloud).
Say in viva: "Grounding means the answer traces back to an actual file and section, not just the model's memory."

Hallucination Mitigation

Definition: techniques that reduce (not eliminate) a model's tendency to state confident but false information.
How it works generally: ground answers in retrieved/real data; refuse to answer when nothing relevant is found.
How LoanMate AI uses it: the 0.65 relevance threshold + explicit "not found" message + deterministic tools for all financial numbers.
Say in viva: "We reduce hallucination risk two ways — real tools for numbers, and a relevance-gated RAG pipeline for knowledge — but we don't claim it's eliminated (see §20)."

Natural Language Generation

Definition: producing fluent, human-readable text/speech as output.
How it works generally: the LLM generates the response token by token, then (here) speech is generated natively by the same audio-native model.
How LoanMate AI uses it: Gemini Live produces both the words and the spoken audio in one pass — no separate, disconnected text-to-speech step on the primary path.
Say in viva: "The reply isn't text converted to speech afterward — Gemini Live generates speech natively, in one audio-in/audio-out session."

17. RAG vs Fine-Tuning

Feature	RAG	Fine-tuning
Knowledge updates	Edit a document + re-run `ingest.py` — minutes	Requires retraining/re-tuning the model on new examples — slow, resource-heavy
Training required	None — the LLM itself is untouched	Yes — model weights are updated
External knowledge	Looked up at query time from a real, inspectable source	"Baked into" model weights, not inspectable per-answer
Cost	Low — an embedding call per chunk once, plus one per query	High — compute-intensive training runs
Best use case	Frequently changing or small, specific knowledge bases	Teaching a model a new skill/style/behavior pattern, not just new facts
LoanMate AI relevance	**Used** — knowledge base is small and expected to change (policies get updated)	**Not used** — no model in this project was fine-tuned; Gemini is used exclusively via its standard API, unmodified

🎤 **SAY THIS — "Why did you choose RAG instead of fine-tuning?"**

"Fine-tuning would mean retraining Gemini on our documents, which is slow, expensive, and — critically — every time a policy changes, we'd have to retrain again. RAG lets us just edit a Markdown file and re-run one indexing script. It's also more transparent: we can see exactly which chunk was retrieved for an answer, which isn't possible with knowledge baked into model weights. For a small, frequently-updatable knowledge base like ours, RAG was the clearly appropriate choice."

18. RAG vs Normal Database Query

Traditional database query	Vector retrieval
`SELECT * FROM docs WHERE category = 'personal_loan'`	"Find text semantically related to what the user is asking"
Requires an exact field/value match or a fixed pattern	Matches based on meaning, regardless of exact wording
Cannot handle "what happens if I miss an EMI" unless that exact phrase (or a hand-coded synonym list) exists as a field	Naturally matches that question to a "Missed EMI Consequences" section, even with zero shared keywords

Why this matters for a natural-language question: a customer will never phrase their question the way a database field is named. Semantic retrieval is valuable precisely because it does not require the user to know the "right" keyword — it only requires the meaning to be close.

> 🤔 **IF TEACHER ASKS WHY — Why not just keyword search (like LIKE '%emi%')?**
>
> Keyword search fails on paraphrases, synonyms, and cross-language questions (a Hinglish question won't literally contain the English keyword). Semantic similarity via embeddings captures meaning, so differently worded — or differently languaged — questions can still match the right section.

19. RAG vs Direct LLM

> 🎤 **SAY THIS — "Why not just send the question straight to Gemini/ChatGPT?"**
>
> "A bare LLM only knows what it learned during training, plus whatever's in the current conversation. It has never seen LoanMate's specific 11 policy documents — so if asked about our prepayment charges, it would have to guess, and that guess could be completely wrong for a financial product. RAG instead retrieves our actual written policy text and gives it to the model before it answers, so the answer is grounded in real project-specific knowledge, is easier to update (just edit a file), and is more traceable — we can see exactly which document and section the answer came from."

Concrete reasons

- **Project-specific knowledge** — a general-purpose LLM has never seen LoanMate's own (fictional demo) policies.
- **Grounding** — the answer is built from real retrieved text, not the model's unverifiable memory.
- **Controllable information** — we decide exactly what's in the knowledge base; nothing outside it can be "recalled."
- **Easier knowledge updates** — edit a Markdown file + re-run `ingest.py`, no retraining.
- **Reduced dependence on pretrained knowledge** — the model doesn't need to "already know" our specific bank's rules.
- **Better traceability** — every RAG answer's source chunk is tagged with file + section.
- **Lower hallucination risk — not zero.**

> ⚠️ **IMPORTANT — do not overclaim**
>
> RAG does **not** guarantee factual correctness. It reduces the *chance* of an unsupported answer by giving the model real material to work from, but the retrieved chunk could still be misapplied, or the model could still phrase something imprecisely. See §20.

20. Hallucination

What is hallucination?

When an LLM states something confidently and fluently that is factually wrong or unsupported — not because it's "lying," but because it generates the most statistically plausible next words, which aren't always true.

Why can LLMs hallucinate?

LLMs are trained to produce fluent, plausible-sounding continuations of text. Fluency and factual correctness are not the same objective — a model can be extremely fluent while being wrong, especially about specific facts it was never actually trained on (like a fictional demo bank's specific prepayment charge percentage).

How does RAG help?

By supplying real, relevant text as part of the model's input before it answers, RAG gives the model something true to draw from, instead of forcing it to invent an answer from vague training-data associations. LoanMate AI adds a further safeguard: the 0.65 relevance threshold means if nothing in the knowledge base is actually relevant, the model is told so explicitly, rather than being handed a low-quality "closest match" to awkwardly narrate.

Does RAG eliminate hallucination?

> ⚠️ **NO — say this precisely**
> "No. RAG reduces the chance of unsupported generation by supplying relevant context, but it does not eliminate hallucination. The system can still retrieve a chunk that's only partially relevant, or the model could still phrase or apply that retrieved information incorrectly. RAG lowers the risk; it does not remove it."

What else does LoanMate AI do to reduce it (separate from RAG)

- Every customer-specific number (EMI, balance, due date) is required to come from a real deterministic tool call — the model is never allowed to compute or state one from memory.
- A verified live test confirmed the agent declining an off-topic question ("what's the weather today") honestly, rather than inventing an answer.

21. Prompting & Grounding

What is a prompt?

The text (instructions + context + question) given to an LLM as its input, which shapes what it generates.

The generic RAG mental model of prompting

Conceptually, a RAG system constructs something like:

```
Context:
[retrieved knowledge chunks]

Question:
[user question]

Instruction:
Answer using the provided context.
```

> 🔍 **VERIFY FROM CODE**
> This exact literal template is **not** claimed to exist verbatim anywhere in LoanMate AI's code — it is the standard conceptual pattern used to explain RAG. Do not tell a teacher "this exact text is in our code" unless you've re-checked `prompts.py` and `realtime_agent.py` yourself.

How LoanMate AI actually delivers retrieved context — an important nuance

LoanMate AI's realtime path does **not** build a fresh "Context: ... Question: ..." text block for a one-shot LLM call. Instead, the retrieved, source-attributed text is returned as the **result of a function/tool call** (`search_loan_knowledge`) inside an *already open*, continuous Gemini Live conversation session. Gemini Live decides to call the tool, receives its string return value as the tool's result, and continues generating its spoken reply with that result now part of the session's context — this is "agentic RAG" delivered via native tool-calling, not a static single-shot prompt template.

> 🎤 **SAY THIS**
> "We don't rebuild a 'Context + Question' prompt from scratch every turn. Retrieval happens through native function/tool calling — the model itself decides to call our search tool, and the retrieved text comes back as that tool's return value inside the same ongoing conversation. That's a more integrated, agentic form of RAG than a plain 'stuff the context into one prompt' pattern."

Why context is included at all — the concept of grounding

Grounding means tying the model's generated output to real, specific, checkable source material, rather than letting it answer purely from its own internal (unverifiable) training knowledge. Including retrieved text — whether as a literal prompt block or as a tool-call result — is what makes the generated answer grounded instead of guessed.

22. Data Flow Diagram

Indexing time (offline, one-time, run manually)

```
backend/knowledge/*.md  (11 files)
      |
      ▼
document_loader.py   (Loader)
      |
      ▼
chunker.py            → 75 Chunks
      |
      ▼
embeddings.py          → embed_document()  (gemini-embedding-2)
      |
      ▼
vector_store.py        → VectorStore.build() + save()
      |
      ▼
loanmate.index + metadata.json   (Vector Store, on disk)
```

Query time (online, every live call, only when RAG is triggered)

```
User
   |
   ▼
Question  (spoken, understood by Gemini Live)
   |
   ▼
Query Embedding  (embeddings.py → embed_query(), async)
   |
   ▼
Vector Search  (vector_store.py → FAISS IndexFlatIP.search())
   |
   ▼
Relevant Chunks  (retriever.py → filtered at ≥ 0.65)
   |
   ▼
Context  (rag_service.py → formatted, source-attributed text)
   |
   ▼
LLM  (Gemini Live receives it as the tool-call result)
   |
   ▼
Response  (natural spoken answer, streamed back to the user)
```

> ⚠️ **IMPORTANT — the distinction that matters most**
>
> Indexing happens **once, offline, manually** (`python -m backend.rag.ingest`), and makes embedding API calls only for the 75 chunks. Querying happens **every time a live call triggers RAG**, and makes exactly one embedding API call (for the user's question) plus one local, fast FAISS search. The running app never re-runs indexing on its own.

23. Technology Stack

Only technologies actually present in the repository are listed below (confirmed via `requirements.txt`, `frontend/web/package.json`, and source imports).

Frontend

Technology	Version	Role
Next.js (primary — `frontend/web/`)	16.3.5	React framework, App Router; the real user-facing voice app
React	19.2.8	UI rendering
TypeScript	^5	Static typing
Tailwind CSS	^4	Styling
@livekit/components-react	^2.9.24	React hooks/components for the LiveKit call (voice orb state, transcript)
livekit-client	^2.22.3	Underlying WebRTC client SDK
Streamlit (legacy — `frontend/app.py`)	≥1.36.0	Older text/voice chat demo + calculator widgets, kept as a fallback reference

Backend

Technology	Version	Role
Python	3.13 (dev environment)	Backend language
FastAPI	≥0.115.0	REST API framework
Uvicorn	≥0.30.0	ASGI server
Pydantic	≥2.0	Request/response validation
livekit-agents[google]	~1.8	LiveKit Agents worker framework + Gemini Live plugin

Generative AI / LLM

Technology	Where used	Role
Gemini Live (`gemini-3.8-live`)	`backend/voice_agent/realtime_agent.py` via `livekit.plugins.google.realtime.RealtimeModel`	**Primary conversational + voice model** — realtime audio in/out, native tool calling
Gemini text (`gemini-3.6-flash`)	`backend/services/llm_service.py` (legacy, deprecated `google-generativeai` SDK)	Legacy text-mode conversation/narration only
Gemini embeddings (`gemini-embedding-2`)	`backend/rag/embeddings.py` via `google.genai.Client` (current SDK)	Turns knowledge chunks and queries into 768-dim vectors for RAG
Function/tool calling	`@function_tool()` decorator (LiveKit Agents SDK)	Lets Gemini Live invoke Python functions mid-conversation

RAG / Vector Search

Technology	Role
FAISS (`faiss-cpu`)	Local, exact (flat) vector index — `IndexFlatIP` , no ANN structure needed at this corpus size
Local FAISS index + JSON sidecar	`backend/rag/vector_store/loanmate.index` + `metadata.json` — no hosted/paid vector database

Realtime Communication

Technology	Role
LiveKit Cloud	Hosts WebRTC rooms, media relay, and agent auto-dispatch
WebRTC	Underlying realtime audio transport standard
LiveKit Agents (`AgentSession`)	Framework wrapping the Gemini Live session, exposing turn-taking/state

Data / Knowledge

Technology	Role
Markdown knowledge base (`backend/knowledge/*.md` , 11 files)	The RAG source content
Flat JSON files (`mock_data.json` , `loan_knowledge.json`)	Demo customer/loan records and the legacy path's own FAQ data — no database, no ORM

Explicitly NOT used

No LangChain/LlamaIndex or agent-orchestration framework (tool-calling is native to the LiveKit Agents SDK). No hosted/paid vector database. No fine-tuning of any model. No non-Google LLM provider. No real telephony (PSTN/Twilio) — outbound/inbound calls are browser-simulated via LiveKit rooms.

24. Why the Project Is Modular

The RAG implementation is deliberately split into six single-purpose files rather than one large script:

```
document_loader.py → chunker.py → embeddings.py → vector_store.py → retriever.py → rag_service.py
```

Why this matters

- **Separation of concerns** — each file does exactly one job (load, split, embed, store, search, orchestrate). No file needs to know how the others do their job internally.
- **Maintainability** — a bug in chunk boundaries is obviously in `chunker.py`; a bug in scoring is obviously in `retriever.py` or `vector_store.py`.
- **Debugging** — you can run any stage in isolation (each file has a small `if __name__ == "__main__":` runner for exactly this) and inspect its output before trusting the next stage.
- **Scalability** — swapping any one piece (a different embedding model, a different vector index type) doesn't require touching the others, as long as the interface (a list of floats in, a list of floats out) stays the same.
- **Testing** — `eval_set.py` tests the retriever in isolation, against the real index, without needing a live voice call.
- **Replaceability** — each stage has a narrow, well-defined interface, so any one implementation can be swapped later.

Concrete "what would change" questions

Change	What needs to change
Switch to a different embedding model	Only `embeddings.py` (and re-run `ingest.py` once, since old vectors would no longer be comparable to new ones)
Switch to a different/hosted vector database	Only `vector_store.py`'s `build/save/load/search` implementation — `retriever.py` calls the same interface either way
Change the document format (e.g. add PDF support)	Only `document_loader.py` — as long as it still returns the same `RawDocument` shape, nothing downstream needs to change
Change chunk size/strategy	Only `chunker.py`
Change the relevance threshold or top_k	Only `retriever.py`

25. Security / Reliability Considerations

Implemented (confirmed in code)

- Backend-side customer validation — an unknown `customer_id` gets HTTP 404 before any token is issued.
- Server-side customer context — the browser never supplies the customer's name or loan ID directly; only `customer_id`, `call_mode`, `outbound_reason`, and `language` are accepted.
- API keys are backend-only — `GOOGLE_API_KEY` and LiveKit credentials live in server-side environment variables, never shipped to the frontend bundle.
- Tool access is customer-scoped — each tool closes over one already-validated customer's data; a tool call can't be redirected to another customer's data mid-conversation.
- Deterministic financial calculations — all EMI/prepayment/eligibility math is plain Python, not model-generated.
- **The RAG knowledge base contains no customer data** — `backend/knowledge/*.md` holds only general demo products/policies/FAQs, never any individual's name, loan ID, or financial figures. Customer data and RAG content are architecturally separate stores.

Recommended future improvement (NOT implemented)

- **No user authentication/login system** exists on any route — any client that can reach the backend can request a token for any demo customer ID.
- **CORS is fully open** (`allow_origins=["*"]`) — fine for local development, not for real deployment.
- **No encryption-at-rest** on the mock data files or the FAISS index — plain, unencrypted files on disk.
- No production-grade audit logging or compliance controls of any kind.
- **Prompt injection** — not specifically defended against in this project; a knowledge document (or a crafted user question) could in principle attempt to influence the model's behavior. This is a known, general RAG/LLM risk class, listed here as a recognized future concern, not something LoanMate AI has implemented a defense for.
- **Input validation** beyond Pydantic's request-schema validation and the customer-existence check — no additional sanitization of free-form question text before it reaches the model.

> ⚠️ **IMPORTANT**
> Be explicit in viva about the Implemented vs Recommended-future-work line above. Claiming an unimplemented safeguard exists is worse than admitting it's a known gap — the project's own documentation is deliberately honest about this distinction, and so should you be.

26. Limitations

- **Knowledge base is small and demo-scoped** — 11 documents, 75 chunks. The pipeline is real and works, but hasn't been evaluated at a scale where an approximate/ANN vector index (HNSW, etc.) would become necessary.
- **RAG quality depends on chunking** — section-based chunking works well because the source documents were deliberately written with clean `##` headings; messier source documents would need a different strategy.
- **Retrieval quality depends on the embedding model** — a weaker embedding model would produce worse similarity matches.
- **Incorrect or poorly-written source documents would produce misleading answers** — RAG grounds the model in whatever is in the knowledge base, correct or not.
- **The LLM can still phrase retrieved information imprecisely** — RAG reduces, but does not eliminate, the risk of an incorrect final answer (§20).
- **No system here is a substitute for real financial advice** — this is an academic/demo project using mock customer data, not a production banking system.
- **Knowledge may become outdated** — updating requires manually editing a file and re-running `ingest.py`; nothing updates itself automatically.
- **RAG has a real, measured cold-start latency** — the first `search_loan_knowledge` call in a session takes roughly 4.8–8.6 seconds (first network connection cost); subsequent calls in the same session are much faster (as low as ~568 ms once warm).
- **No formal, large-scale evaluation framework** — the RAG evaluation set (`eval_set.py`) covers 8 test cases, which all pass, but this is not a large, statistically rigorous precision/recall study.
- **No automated test coverage for the realtime voice path's prompt/tool behavior** — verification has been manual/scripted, not committed pytest tests (the 36 committed pytest tests cover only the legacy REST/orchestrator layer).

27. Future Scope

Improvement	Why it would help
Larger knowledge base	Cover more real products/edge cases; would justify moving to an ANN index (HNSW/IVF) for speed at scale
Metadata filtering	Let retrieval be restricted to, e.g., only `policies/` when the question is clearly policy-related, improving precision
Improved chunking	Handle source documents that aren't as cleanly structured as the current 11 files
Hybrid search (keyword + vector)	Combine exact-term matching with semantic search for queries containing precise terms (loan IDs, exact figures)
Reranking	A second-pass model to re-order retrieved chunks by finer-grained relevance before generation
Source citations in the spoken/UI response	Surface the `[source: file — section]` tag (already computed internally) to the end user, not just to logs
Adaptive confidence/relevance thresholds	Tune 0.65 dynamically per query type instead of one fixed global value
Conversation memory across calls	Currently every call is a fresh, stateless session — persistent memory would let LoanMate recall earlier conversations
Real user authentication	Close the "no login system" gap noted in §25
Multilingual expansion	Beyond English/Hindi/Hinglish, to more Indian regional languages
Document upload / OCR	Let new policy PDFs or scanned documents be ingested directly instead of hand-written Markdown
Formal evaluation framework	A larger, labeled question set with precision/recall metrics, beyond the current 8-case eval set
Monitoring / observability	Structured logging and tracing for production operation
Guardrails	Explicit defenses against prompt injection and other LLM-specific attack classes
Human escalation	A clear "talk to a human" handoff path (a narrow keyword-based version exists only in the legacy pipeline today)
Advanced financial-product comparison	Deeper "what-if" tooling beyond the current prepayment/tenure/eligibility calculators
Eliminate RAG cold-start latency	Issue a warm-up embedding call at worker startup instead of on the first real question

28. Possible Teacher Questions — Basic

1. What is LoanMate AI?

A voice-first conversational loan assistant that lets customers ask about EMI, balance, and loan policies in English, Hindi, or Hinglish, powered by Gemini Live and a RAG knowledge base.

2. What problem does it solve?

It replaces multi-step dashboard navigation with a single spoken question, and avoids the risk of an ungrounded AI guessing at policy answers.

3. Why did you choose this problem?

Loan/EMI queries are simple in content but tedious to find through a typical banking UI — a good fit for demonstrating both agentic tool-calling and RAG in one coherent, realistic domain.

4. What is Generative AI?

AI that creates new content — here, new natural-language sentences — rather than selecting from a fixed set of pre-written answers.

5. Where is AI used in your project?

Gemini Live for the entire conversation and tool-calling decisions; a separate Gemini embedding model for RAG.

6. What is RAG?

Retrieval-Augmented Generation — retrieving relevant real text from a knowledge base before the LLM generates its answer.

7. Why did you use RAG?

So general knowledge/policy answers are grounded in our real 11 documents instead of the model's own unverifiable guesses.

8. What is an embedding?

A numeric vector representing a piece of text's meaning, so similar meanings end up as nearby vectors.

9. What is a vector?

An ordered list of numbers — 768 numbers, in this project's embeddings.

10. What is a vector store?

A database optimized to store vectors and quickly find the ones most similar to a given query vector — here, a local FAISS index.

11. What is semantic search?

Finding text by meaning rather than exact keyword matching.

12. What is chunking?

Splitting a document into smaller, topically focused pieces before embedding them.

13. Why do you split documents?

So each embedding represents one coherent topic, making retrieval precise instead of blurry.

14. What is retrieval?

Searching the vector store for the chunks most relevant to a question.

15. What is hallucination?

When an LLM confidently states something false or unsupported.

16. How does RAG reduce hallucination?

By giving the model real retrieved text to work from, and by refusing to answer (via a relevance threshold) when nothing relevant is found — but it doesn't eliminate hallucination entirely.

17. What happens when a user asks a question?

Gemini Live decides whether it needs a deterministic tool, the RAG tool, or can answer directly from context, then generates a natural spoken reply.

18. What is an LLM?

Large Language Model — a large pretrained neural network that understands and generates language.

19. What is prompting?

The instructions/context given to an LLM to shape its output.

20. What is context?

The information (customer data, retrieved chunks, conversation history) available to the model when generating a reply.

21. What is the knowledge base?

11 Markdown files under `backend/knowledge/` covering loan products, policies, FAQs, and required documents.

22. Why Markdown files?

Their `##` section headings give the chunker a natural, semantic way to split each document into topically coherent pieces.

23. Why not use only a normal database?

A database matches exact field values; it can't match a free-form natural-language question to the right paragraph by meaning.

24. What happens if the answer isn't in the documents?

The relevance threshold (0.65) filters out irrelevant chunks, and the system returns an explicit "no relevant information found" message rather than inventing an answer.

25. What are the limitations?

Small demo knowledge base, cold-start RAG latency, no guarantee of full factual correctness, mock data only — see §26 for the full list.

29. Possible Teacher Questions — Technical

1. How are documents loaded?

`document_loader.py` recursively finds every `.md` file under `backend/knowledge/`, reads it, and tags it with folder-derived `document_type` and filename-derived `topic`.

2. Why chunk documents?

So each embedding represents one focused topic, improving retrieval precision, and so the LLM only receives small, relevant pieces instead of whole documents.

3. How are chunks converted into embeddings?

`embeddings.py` calls the Gemini embedding model (`gemini-embedding-2`) once per chunk, with `task_type="RETRIEVAL_DOCUMENT"`, truncated to 768 dimensions.

4. What does an embedding represent?

The semantic meaning of the input text, as a 768-number vector.

5. How is similarity calculated?

Cosine similarity, implemented as an inner product over L2-normalized vectors in FAISS's `IndexFlatIP`.

6. What does the retriever return?

Up to `top_k` (default 3) chunks scoring ≥ 0.65 cosine similarity, each with text, score, and source metadata.

7. What is top-k retrieval?

Returning only the k highest-scoring matches instead of the entire knowledge base — here, k=3.

8. What is context injection?

Placing known information (like the retrieved chunks, or the customer's name) directly into the model's available input.

9. How does the RAG service orchestrate components?

`rag_service.py` lazily loads a `Retriever` singleton, calls `retrieve()`, and formats the result (or an explicit no-match message) for the calling tool.

10. What happens during indexing?

Load → chunk → embed each chunk (one call per chunk, ~630 ms/chunk average, 47.2 s total for 75 chunks) → build a FAISS index → save two local files.

11. What happens during query time?

Embed the query (async, RETRIEVAL_QUERY task type) → search the pre-built FAISS index → filter by threshold → format and return.

12. Why separate indexing and retrieval?

Indexing is a slow, one-time, offline cost (embedding 75 chunks); retrieval must be fast and happen live during a call. Keeping them separate means normal app operation makes zero embedding calls except for the live query itself.

13. What happens if the vector store has irrelevant chunks?

They would score low against a genuinely relevant query and simply not be in the top-k, or would be filtered out by the 0.65 threshold if they somehow scored higher than expected.

14. What happens if retrieval returns no useful results?

`rag_service.py` returns an explicit "no sufficiently relevant information found" message instead of the model narrating a weak or wrong match.

15. How would you evaluate retrieval quality?

Currently via `eval_set.py` — 8 hand-picked cases checking whether the correct source file is the top result. At larger scale, this would need to become a bigger, labeled precision/recall study.

16. How would you improve retrieval?

Metadata filtering, hybrid keyword+vector search, and reranking are all realistic next steps (§27) — none are implemented today.

17. What is reranking?

A second-pass model that re-scores an initial set of retrieved candidates more precisely before generation — not implemented in this project.

18. What is hybrid search?

Combining traditional keyword/exact-term matching with vector similarity search — not implemented in this project.

19. What is metadata filtering?

Restricting a similarity search to a subset of chunks based on metadata (e.g. only `document_type == "policies"`) — the metadata exists on every chunk today, but is not currently used to filter searches, only to attribute sources.

20. What is prompt injection?

An attack where malicious text (in a user question or, in principle, a poisoned knowledge document) tries to override the model's instructions. Not specifically defended against in this project — a known limitation.

21. How can RAG systems be attacked?

Prompt injection via crafted queries or poisoned source documents, and data leakage if sensitive data were mixed into the retrievable knowledge base (not the case here — the RAG knowledge base intentionally contains no customer data).

22. How would you secure this application?

Add real authentication/authorization, restrict CORS, encrypt data at rest, add input validation/guardrails against prompt injection, and add audit logging — all currently unimplemented (§25).

23. How would you scale the vector store?

Move from FAISS's flat/exact `IndexFlatIP` to an approximate/ANN index (HNSW or IVF) once the corpus is large enough that exact search stops being fast — not necessary at the current 75-chunk scale.

24. What happens if the knowledge base grows to 100,000 documents?

Exact search would become too slow; an ANN index would be needed, along with likely metadata filtering/hybrid search to keep precision high.

25. How would you update documents?

Edit the relevant `.md` file, then re-run `python -m backend.rag.ingest` to rebuild the index.

26. How would you handle outdated information?

There's no automatic staleness detection today — it would require a manual review process or a "last updated" metadata field and periodic re-indexing.

27. How would you test the system?

`eval_set.py` for retrieval correctness (8/8 passing), plus manual/scripted simulated-conversation tests for the full agentic + RAG behavior (documented in the project's testing section).

28. How would you measure answer quality?

Currently qualitative (manual review of transcripts). A formal approach would need labeled correct answers and either human grading or an LLM-as-judge evaluation — not implemented here.

29. How would you reduce hallucinations further?

Reranking, stricter/adaptive thresholds, source-citation surfacing to the user, and a larger, better-curated knowledge base.

30. What would you change if the system gave irrelevant answers?

First check the relevance threshold and chunking boundaries; if a whole topic is missing, add a new knowledge document and re-index; if the embedding model is the weak link, that's the only file (`embeddings.py`) that would need to change.

30. "Teacher Trap Questions" — Where Students Commonly Get Stuck

🚨 **"Is RAG itself Generative AI?"**

No. RAG is a *technique* that combines retrieval (search — not generative) with generation (the LLM writing the final answer). Retrieval alone produces no new text; it just finds existing text. RAG only counts as part of a Generative AI *system* because its output feeds a generative model.

🚨 **"Are embeddings generated by the LLM?"**

No. Embeddings are produced by a separate **embedding model** (`gemini-embedding-2` here) — a different model from the conversational LLM (`gemini-3.8-live`). They're both Google/Gemini models, but they're distinct models doing distinct jobs.

🚨 **"Does vector search generate the answer?"**

No. Retrieval finds relevant information (existing chunk text); the generative model (Gemini Live) is what actually produces the natural-language answer.

🚨 **"Does RAG train the LLM?"**

No. RAG retrieves external context at query/inference time. It does not change the model's weights at all — nothing about Gemini itself is modified.

🚨 **"Does RAG eliminate hallucination?"**

No. It reduces the risk by grounding answers in real text, but the model can still misapply or misphrase retrieved information, or retrieval could return a marginally-relevant chunk.

🚨 **"Is a vector database the same as a normal database?"**

No. A normal (relational) database matches exact field values; a vector database (here, FAISS) finds the nearest vectors by similarity, enabling meaning-based matching instead of exact-match lookups.

🚨 **"Why can't keyword search do this?"**

Keyword search needs the exact word/phrase to appear. A Hinglish question with zero English keywords in common with the document still needs to match — that's only possible with semantic (meaning-based) similarity, not literal keyword matching.

🚨 **"What happens before the user asks a question?"**

Indexing — the offline pipeline (load → chunk → embed → build FAISS index → save) that must run at least once before any retrieval is possible.

🚨 **"What happens after retrieval?"**

The retrieved chunks are handed to the LLM as context (via a tool-call result in this project), and the LLM generates the final natural-language answer.

🚨 **"So the LLM doesn't know anything about your bank at all on its own?"**

Correct — Gemini has no built-in knowledge of a fictional bank called "LoanMate." Any specific policy fact it states must come from either a deterministic tool (customer data) or the RAG knowledge base (general policy/product data). Anything else would be an unverifiable guess.

🚨 "Is RAG always used, on every question?"

No. It's one of several tools Gemini Live can choose to call. Customer-specific questions use the 6 data-lookup tools or 3 calculators instead; identity questions ("what's my name") are answered directly from context already in the prompt, with no tool call at all.

🚨 "Isn't the vector store just another database?"

Technically yes — it's a specialized database. But "database" in the traditional sense (as most students mean it) implies exact-match/structured queries; a vector store's defining feature is approximate/exact *similarity* search over numeric vectors, which is a fundamentally different query model.

🚨 "If retrieval finds the wrong chunk, does the system know it's wrong?"

No — the system only knows the chunk's *similarity score*, not whether it's factually the best answer. A high-scoring but subtly wrong chunk would still be handed to the model. This is exactly why the honest answer to "does RAG guarantee correctness" is no.

🚨 "Could you have used a real production vector database like Pinecone instead?"

Yes, conceptually — the interface (`build/save/load/search`) in `vector_store.py` is the boundary that would need to change. At only 75 chunks, a hosted/paid vector database would be unnecessary overhead; the local FAISS flat index is the right-sized choice here.

🚨 "Does the embedding model 'understand' language the way the LLM does?"

Not in the same sense. An embedding model's only job is to output a vector representing meaning for similarity comparison — it does not generate text, hold a conversation, or reason about instructions the way the conversational LLM does.

31. Demo Script

Before demo

> 🎤 **SAY THIS**
> "I'll show LoanMate AI live. I'll click 'Call LoanMate', speak a question naturally, and you'll see/hear the response. Behind the scenes, the system is deciding — per question — whether to look up my account data, run a calculation, or search our knowledge base using RAG."

During demo

1. Open the app, click **Call LoanMate** (inbound).
2. Allow microphone access; wait for "Connected/Listening."
3. Ask the example question below and let the agent respond fully.

Example question (demonstrates RAG clearly)

> ⚠️ **EXAMPLE QUERY**
> "What are the eligibility requirements for a personal loan?"

What's happening behind the scenes while the answer generates: Gemini Live recognizes this is not customer-specific → calls `search_loan_knowledge` → the question is embedded → FAISS is searched → chunks from `eligibility_policy.md` / `personal_loan.md` scoring above 0.65 are retrieved → that real text is handed back to the model → the model composes a spoken answer grounded in it.

After the answer appears

> 🎤 **SAY THIS**
> "This answer is not simply generated from the model's general knowledge. The system first retrieves relevant information from our knowledge base and then uses that context to generate the response — so it's grounded in our own documents, not the model's guess about a generic bank."

3–5 other good demo questions

Question	What it demonstrates
"What is my EMI?"	Deterministic tool call (`get_emi`) — customer-specific, no RAG
"What documents do I need for a personal loan?"	RAG → `required_documents.md`
"Prepayment karne pe koi charges hain kya?" (Hinglish)	RAG working across languages → `prepayment_policy.md`
"Agar main do lakh prepay karun toh kya hoga, aur kya koi charges hain?"	Hybrid: both a calculator tool **and** RAG fire together for one compound answer
"What's the weather today?"	Off-topic — demonstrates the relevance threshold: agent honestly declines instead of guessing

Questions that demonstrate RAG most clearly

Any question about *policy/product/documents in general* (not this customer's account) is a RAG question: foreclosure, missed EMI consequences, required documents, general FAQs. Contrast one of these directly against "what is my EMI" in the demo to visibly show the routing difference to the teacher.

32. Live Architecture Explanation Script (2–3 minutes)

🎤 SAY THIS — while pointing at the architecture diagram

"First, we have our user interface — a Next.js web app with a voice orb. The customer clicks 'Call LoanMate' and speaks into their browser microphone.

That audio travels over WebRTC to LiveKit Cloud, which is our real-time media transport layer. LiveKit automatically dispatches a separate, always-running Python worker process — our Realtime Agent — into that call's room.

That worker opens a single continuous session with Gemini Live, Google's realtime multimodal model. Audio streams directly into Gemini Live, and Gemini Live streams audio directly back out — there's no separate speech-to-text or text-to-speech step on this path; one model does both understanding and speaking.

Now here's the important part — Gemini Live doesn't just talk. On every turn, it decides whether it needs external information. If the question is about this specific customer's account — their EMI, their balance — it calls one of six deterministic Python tools that read a real customer record. If the question is a calculation — prepayment impact, tenure comparison — it calls one of three calculator tools that do plain arithmetic. Neither of those ever involves the AI inventing a number.

But if the question is a general knowledge or policy question — not tied to one customer's account — Gemini Live calls a tenth tool: search_loan_knowledge. That's our RAG pipeline. It embeds the question into a 768-dimensional vector, searches a local FAISS vector index built from 75 chunks of our 11 knowledge documents, and — only if the best matches score above a 0.65 relevance threshold — returns that real text back into the conversation. Gemini Live then uses that retrieved text to generate a grounded, natural-language answer.

Whichever path it took, the final spoken response streams back through LiveKit to the browser, and the customer hears it — and can interrupt it mid-sentence at any point, because Gemini Live processes audio continuously rather than waiting for a full question to finish."

Then explain the indexing side

🎤 SAY THIS

"Separately, and completely offline, there's a one-time indexing pipeline. Whenever I add or edit a knowledge document, I run one command — python -m backend.rag.ingest. It loads every markdown file, splits each one into chunks along its own section headings, sends each chunk to a Gemini embedding model, and saves the resulting vectors into a local FAISS index file on disk. The live application never rebuilds this index itself — it only ever loads the pre-built file. That separation is what keeps normal conversations fast and free of unnecessary API calls."

33. Explain This Project to a Non-Technical Teacher — The Library Analogy

RAG concept	Library analogy
Knowledge documents (11 .md files)	The books in the library
Chunking	Dividing each book into labeled sections/chapters instead of one giant unbroken block of text
Embeddings	Giving each section a meaning-based numerical "index card" — cards for similar topics end up filed near each other, even if worded differently
Vector store (FAISS index)	The organized, intelligent card catalog / index of the whole library
Retriever	The librarian — takes your question, walks to the index, and pulls out the 2–3 most relevant sections
LLM (Gemini Live)	A knowledgeable assistant who reads exactly those pulled-out sections and explains them to you in plain conversation, in your own language
RAG (the whole process)	The librarian finds the correct book section **before** the assistant explains anything — the assistant never just answers from memory when a real book has the answer
Relevance threshold (0.65)	The librarian says "I don't have a book that actually covers that" instead of handing over an unrelated book and pretending it answers your question

🎤 **SAY THIS — full analogy, ready to speak**

"Think of it like a smart library. We have 11 books. Each book is split into labeled sections. Every section gets a 'meaning fingerprint' — sections about similar things have similar fingerprints, even if the words are different. When you ask a question, a librarian compares your question's fingerprint to every section's fingerprint and pulls out the two or three closest matches — but only if they're actually close enough to be useful. Then, instead of just handing you the raw book pages, an assistant reads those sections and explains the answer to you conversationally, in your own words and language. That whole process — find the right pages first, then explain — is what RAG means."

34. Important Terminology Cheat Sheet

Term	One-line definition
LLM	Large Language Model — a pretrained neural network that understands and generates language
GenAI	Generative AI — AI that creates new content (text, speech, images) rather than selecting from fixed options
RAG	Retrieval-Augmented Generation — retrieve relevant real text, then generate an answer grounded in it
Embedding	A numeric vector representing the meaning of a piece of text
Vector	An ordered list of numbers (768 of them, in this project)
Vector Store	A database optimized for finding the most similar stored vectors to a query vector (here: local FAISS)
Retriever	The component that embeds a query, searches the vector store, and filters by relevance
Chunk	One small, topically focused piece of a larger document
Chunking	The process of splitting a document into chunks
Semantic Search	Finding text by meaning rather than exact keyword match
Prompt	The instructions/context/question given to an LLM as input
Context	All the information (data, retrieved text, history) available to the model when it generates a reply
Grounding	Tying a generated answer to real, checkable source material
Hallucination	An LLM confidently stating something false or unsupported
Inference	Running a trained model to produce an output (as opposed to training it)
Knowledge Base	The structured collection of source documents a system can retrieve from — here, 11 Markdown files
Similarity Search	Searching by closeness in vector space (here: cosine similarity)
Top-K	Returning only the K highest-scoring results (here, K=3)
Metadata	Extra descriptive fields attached to a chunk (source_file, document_type, topic, section)
Reranking	A second, finer-grained relevance pass over an initial set of retrieved results — not implemented here
Fine-tuning	Retraining a model's weights on new examples — not used anywhere in this project
Inference vs Training	Training changes model weights (not done here); inference just runs the already-trained model on new input (all of LoanMate's model usage is inference)

35. 30-Second Answers

"What is RAG?"

"Retrieval-Augmented Generation. Before answering a knowledge question, the system embeds the question, searches a vector index of our own documents, and gives the matched text to the LLM so the answer is grounded in real material instead of a guess."

"What is an embedding?"

"A numeric vector that represents a piece of text's meaning, so texts with similar meaning end up with similar vectors — even if the wording is completely different."

"What is a vector store?"

"A database built to quickly find the most similar stored vectors to a given query vector. We use a local FAISS index with 75 vectors, one per knowledge chunk."

"What is Generative AI?

"AI that generates new content — in our case, Gemini Live generates a brand-new natural-language sentence every single reply, rather than picking from a fixed script."

"Where is GenAI used?"

"Gemini Live drives the entire conversation and decides on tool calls; a separate Gemini embedding model powers our RAG search."

"Why RAG?"

"So general policy/product answers are grounded in our actual 11 documents, are easy to update by editing a file, and are traceable to a real source — instead of the model guessing."

"Why not fine-tuning?"

"Fine-tuning means retraining the model — slow, expensive, and it would need repeating every time a policy changes. RAG just needs a file edit and a re-index."

"What is hallucination?"

"When an LLM confidently states something false. RAG reduces this risk by grounding answers in real text, but doesn't eliminate it."

"How does your system work?"

"The customer speaks; Gemini Live decides whether it needs a customer-data tool, a calculator tool, or our RAG tool; whichever fires, it generates a natural spoken reply using the real result."

"What happens when a user asks a question?"

"Gemini Live reasons about the question, optionally calls a tool to get real data or retrieve knowledge, then generates a grounded spoken answer."

"What happens before the user asks a question?"

"Offline indexing — our 11 documents are loaded, split into 75 chunks, embedded, and stored in a FAISS index, ready to be searched."

"What does the retriever do?"

"Embeds the question, searches the FAISS index for the top 3 matches, and discards anything scoring below a 0.65 relevance threshold."

"What does the LLM do?"

"Generates the actual natural-language spoken answer — it's the only component that writes new text; retrieval just finds existing text."

"What does the vector store do?"

"Stores every knowledge chunk's embedding and lets us search for the most similar ones to a given query, fast."

"What does the knowledge base contain?"

"11 Markdown documents — loan products, policies, FAQs, and required documents — written as fictional demo content for LoanMate Bank."

36. Final Presentation Story

1. Problem

"Loan/EMI information is simple in content but tedious to access — buried behind logins, menus, and dashboard widgets, especially difficult for customers more comfortable speaking than navigating an English-only UI."

2. Solution

"LoanMate AI replaces that navigation with a live spoken conversation, powered by an agentic Generative AI model that decides — per question — whether it needs real account data, a calculation, or our own knowledge base."

3. Architecture

"A Next.js voice UI talks to a FastAPI backend and LiveKit Cloud, which dispatches a dedicated worker process holding a live Gemini Live session — one continuous audio-in/audio-out connection, not a record-transcribe-reply cascade."

4. Technology

"Next.js/React frontend, FastAPI backend, LiveKit for realtime WebRTC, Gemini Live for conversation, a separate Gemini embedding model plus a local FAISS index for RAG."

5. GenAI

"Generative AI is Gemini Live itself — every reply is dynamically generated language, not a pre-written template, grounded either in real tool data or retrieved knowledge."

6. RAG

"For general knowledge questions, we retrieve from our own 11-document knowledge base — chunked into 75 sections, embedded, stored in FAISS, and searched with a 0.65 relevance threshold — before the model answers."

7. Demo

"I'll show a live call: one customer-data question, one RAG question, and one hybrid question that uses both at once."

8. Limitations

"Small demo knowledge base, real cold-start latency on the first RAG call, mock customer data, no production authentication — all stated honestly, not hidden."

9. Future scope

"A larger knowledge base with an ANN index, reranking, source citations in the UI, real authentication, and a formal evaluation framework."

10. Conclusion

> 🎤 **SAY THIS — closing line**
> "LoanMate AI uses a RAG pipeline to retrieve relevant information from a structured loan knowledge base and uses a generative AI model to produce a natural-language answer grounded in that retrieved context — combined with agentic tool-calling so that every financial figure comes from real data, never from the model's imagination."

37. Final Cheat Sheet — Before Entering Class

Project

LoanMate AI — voice-first loan assistant, English/Hindi/Hinglish

Core concept

Generative AI (Gemini Live, agentic tool-calling) + Retrieval-Augmented Generation (RAG)

Knowledge

11 Markdown documents, 75 chunks

Knowledge categories

loan_products/, policies/, faq/, documents/

RAG modules (backend/rag/)

`document_loader.py · chunker.py · embeddings.py · vector_store.py · retriever.py · rag_service.py` (plus `ingest.py` for offline indexing, `eval_set.py` for testing)

Indexing (offline, one-time)

Documents → Load → Chunk (75) → Embed (gemini-embedding-2, 768-dim) → Store (FAISS IndexFlatIP)

Query (online, live call)

Question → Embed → Retrieve (top_k=3, ≥0.65 threshold) → Context → LLM (Gemini Live, via tool call) → Answer

Key distinction

The retriever finds information. The LLM (Gemini Live) generates the response. Retrieval is search; generation is Generative AI.

Key benefit

Grounded responses using project-specific knowledge, traceable to a real file + section.

Key limitation

RAG reduces hallucination risk — it does not eliminate it.

RAG is a tool, not a rule

Gemini Live only calls `search_loan_knowledge` for general knowledge questions — customer-specific facts go through 6 deterministic data tools; calculations go through 3 calculator tools.

Numbers to remember

11 docs · 75 chunks · 768 dimensions · top_k=3 · threshold=0.65 · gemini-embedding-2 · gemini-3.8-live · FAISS IndexFlatIP

38. Details I Must Verify From My Code Before Presentation

> 🔍 **VERIFY THESE BEFORE THE VIVA**
>
> - The exact literal wording inside each of the 11 knowledge documents (specific interest rates, exact charge percentages, exact eligibility numbers) — re-open the actual files if the teacher asks for a specific figure.
> - The exact literal text of the system prompt(s) in `backend/voice_agent/prompts.py` — this guide describes its structure and rules, not a verbatim quote of every sentence.
> - The exact contents of `call_context.py` and the frontend component file names (`page.tsx` , `Home.tsx` , `ActiveCall.tsx` , `OutboundFlow.tsx`) if asked to walk through the frontend file structure in detail.
> - Whether the currently deployed `.env` / environment has a real, working `GOOGLE_API_KEY` at presentation time — if the key is invalid or quota-limited, the live demo's embedding/Gemini Live calls will fail even though the code is correct.
> - Whether the FAISS index files (`backend/rag/vector_store/loanmate.index` , `metadata.json`) currently exist and are up to date with the latest knowledge documents — if any `.md` file was edited after the last `ingest.py` run, re-run it before presenting.
> - Whether the LiveKit Agents worker process (`python -m backend.voice_agent.realtime_agent dev`) is actually running alongside the FastAPI backend during the live demo — the RAG tool only fires inside a live call, which requires this separate worker process.
> - Any project-specific number not explicitly stated in this guide (e.g., exact latency on presentation-day hardware/network, which will differ from the previously measured 4.8–8.6s cold-start / ~568ms warm-call figures).

Every other number in this guide (11 documents, 75 chunks, 768 dimensions, top_k=3, 0.65 threshold, model names, tech stack versions) was cross-checked directly against the project's source code and its own existing verified documentation at the time this guide was written — not invented.

Mock Viva — 20+ Exchanges (Basic → Technical, With Follow-ups)

Teacher: "In one line, what is LoanMate AI?"
Student: "A voice-first loan assistant that answers account questions using real data and general policy questions using a RAG pipeline over our own documents."

Teacher: "Where exactly is Generative AI in your project?"
Student: "Gemini Live — it generates every spoken reply dynamically, and it decides, per turn, whether to call a tool."

Teacher: "But isn't retrieval just search?"
Student: "Yes — retrieval by itself is semantic search, not generation. It only finds existing chunks."

Teacher: "Then what actually generates the answer?"
Student: "Gemini Live. It receives the retrieved chunks as a tool result and writes a brand-new sentence using them."

Teacher: "What is an embedding, exactly?"
Student: "A vector of 768 numbers representing the meaning of a piece of text, produced by our embedding model, gemini-embedding-2."

Teacher: "Is that the same model that talks to the customer?"
Student: "No — gemini-embedding-2 only produces vectors, it never generates conversational text. The conversational model is a separate one, gemini-3.8-live."

Teacher: "How do you decide if a retrieved chunk is good enough to use?"
Student: "A cosine similarity score, and we discard anything below 0.65."

Teacher: "Where did 0.65 come from? Did you just guess it?"
Student: "No — we measured real scores: on-topic questions scored 0.713 to 0.860, off-topic questions scored 0.525 to 0.591. 0.65 sits in that gap."

Teacher: "What if a question is both a policy question and needs a calculation?"
Student: "We verified a hybrid case live — 'if I prepay two lakh, what happens and is there a charge' — the agent called both the prepayment calculator and search_loan_knowledge, and combined both real results into one answer."

Teacher: "What database do you use for the vectors?"
Student: "FAISS, running locally — specifically IndexFlatIP, an exact/flat index, not an approximate one."

Teacher: "Why not an approximate index like HNSW? Isn't that more advanced?"
Student: "At only 75 vectors, exact search is already sub-millisecond. An approximate index would trade accuracy for a speed benefit we don't need at this scale — so exact search is actually the more correct choice here, not a less advanced one."

Teacher: "What happens if I ask something not in your documents at all?"
Student: "The similarity scores stay below 0.65, so the retriever returns nothing, and the system returns an explicit 'no relevant information found' message instead of a guess."

Teacher: "Does that guarantee the model won't hallucinate?"
Student: "No. It stops the model from being handed an irrelevant chunk to narrate, but it can't guarantee the model phrases a genuinely relevant chunk perfectly. RAG reduces hallucination risk, it doesn't eliminate it."

Teacher: "Why didn't you fine-tune Gemini on your documents instead?"
Student: "Fine-tuning requires retraining every time a policy changes, is expensive, and isn't transparent about which fact came from where. RAG just needs a file edit and re-running one indexing script, and every answer is traceable to a specific source chunk."

Teacher: "How many documents and chunks do you actually have?"
Student: "11 Markdown documents, producing 75 chunks after section-based splitting."

Teacher: "Why did you split by markdown headings instead of, say, every 500 characters?"
Student: "Our documents were written with meaningful section headings — Overview, Eligibility, Charges. Splitting on those keeps each chunk topically coherent and gives us a real section name to attach as metadata, instead of a blind character cut that could split a sentence in half."

Teacher: "Does your chunker overlap chunks, like many RAG tutorials recommend?"
Student: "No — our chunks are non-overlapping sections. Overlap wasn't necessary because each markdown section is already a coherent, self-contained unit."

Teacher: "How is the retrieved information actually given to the model — do you build a big prompt?"
Student: "Not a static prompt template. It's delivered as the return value of a function/tool call — search_loan_knowledge — inside the already-open Gemini Live session. That's agentic RAG through native tool-calling, not a rebuilt Context-plus-Question prompt every turn."

Teacher: "What's the difference between your 9 other tools and this RAG tool?"
Student: "The other 9 either read one specific customer's real data or run deterministic math. search_loan_knowledge never touches customer data at all — it only searches a small, general, read-only knowledge base."

Teacher: "Is there any risk the RAG tool could leak another customer's data?"
Student: "No — architecturally, the knowledge base contains zero customer data. Customer information lives only in mock_data.json, accessed only by the other 9 tools, which are scoped to one validated customer per call."

Teacher: "What's slow about your RAG pipeline, if anything?"
Student: "The first search_loan_knowledge call in a session has a real, measured cold-start cost of about 4.8 to 8.6 seconds, from the embedding client's first network connection. Later calls in the same session are much faster — as low as around 568 milliseconds."

Teacher: "Could you fix that cold-start delay?"
Student: "Yes — a realistic future improvement is issuing a warm-up embedding call when the worker starts, rather than on the first real user question. That's proposed future work, not implemented yet."

Teacher: "If I gave you 10,000 more documents tomorrow, what would break?"
Student: "The exact FAISS index would get slower to search linearly with size, so at that scale I'd switch vector_store.py to an approximate index like HNSW. I'd also likely want metadata filtering and possibly reranking to keep precision high with so much more content to search through."

Teacher: "Last question — in one sentence, what does your project actually prove you understand?"
Student: "That I can combine agentic tool-calling for verified factual data with a real, working retrieval-augmented generation pipeline for general knowledge — and that I understand exactly where each one is appropriate, and where each one's limits are."

If You Remember Only 10 Things

1. LoanMate AI is a voice-first loan assistant using Gemini Live + agentic tool-calling + RAG.
2. Generative AI = Gemini Live generating natural spoken language every turn. Retrieval ≠ generation; retrieval only finds text.
3. RAG is used only for general knowledge/policy questions — customer facts and calculations go through 9 separate deterministic tools.
4. Knowledge base: 11 Markdown files → chunked into 75 sections (split on `##` headings, not blind character cuts).
5. Embeddings: `gemini-embedding-2`, 768 dimensions, separate task types for documents vs. queries.
6. Vector store: local FAISS `IndexFlatIP` — exact cosine similarity search, no hosted database.
7. Retriever returns top_k=3, filtered at a relevance threshold of 0.65 (empirically measured, not guessed).
8. Indexing is offline/one-time (`ingest.py`); querying happens live, only when RAG is actually triggered.
9. RAG reduces hallucination risk — it does **not** eliminate it. Never claim otherwise.
10. **LoanMate AI uses a RAG pipeline to retrieve relevant information from a structured loan knowledge base and uses a generative AI model to produce a natural-language answer grounded in that retrieved context.**

Good luck. You built something real — walk in and explain it like you did.